Routage optimal dans les écosystèmes AMM
(Optimal Routing in AMM-based Ecosystems)

Ter (M1) — Encadrant : Sebastian Müller
sebastian.muller@univ-amu.fr

Résumé

Ce mémoire présente une synthèse structurée de la modélisation mathématique du routage optimal dans les CFMM, explique la relaxation convexe du problème, puis étudie l'interprétation de l'arbitrage comme cas particulier du routage.

Sommaire

1 Introduction **2**
1.1 DEX, limites des carnets d'ordres on-chain et rôle des AMM . . . 2
1.2 Le cas Uniswap V2 : invariant de produit constant . . . 2
1.3 Prix marginal, prix effectif et slippage . . . 3
1.4 Arbitrage local et efficacité des prix . . . 3
1.5 Du pool isolé au réseau de pools : motivation du routage optimal . . . 4

2 Modélisation Mathématiques d'un réseau de CFMM **4**
2.1 Réseau de m CFMM et n tokens . . . 4
2.2 Indexation locale/globale et matrices A_i . . . 6
2.3 Paniers fournis et reçus : Δ_i et Λ_i . . . 7
2.4 Fonction de trading φ_i et frais γ_i . . . 8
2.5 Vecteur de transaction réseau Ψ . . . 9
2.6 Fonction d'utilité du trader . . . 10

3 Problème de routage optimal **11**
3.1 Formulation avec contraintes d'égalité . . . 11
3.2 Relaxation convexe . . . 12
3.3 Exactitude de la relaxation . . . 14
3.4 Exemple : liquidation d'un token vers un autre . . . 14
3.5 Extension : coûts fixes et problème MICP . . . 14

4 Arbitrage et équilibre du marché **14**
4.1 Définition d'un arbitrage . . . 14
4.2 Détection d'arbitrage par routage optimal . . . 14
4.3 Conditions de non-arbitrage . . . 14
4.4 Interprétation : prix globaux, prix locaux et bande de non-arbitrage . . . 14
4.5 Limites économiques : frais, liquidité, gaz, exécution . . . 14

5 Application empirique et backtesting sur données réelles **14**

Bibliographie **15**

1 Introduction

Les échanges décentralisés, ou *Decentralized Exchanges* (DEX), constituent l'une des applications les plus importantes de la finance décentralisée. Ils permettent à des utilisateurs d'échanger des actifs numériques sans passer par un intermédiaire centralisé chargé de conserver les fonds ou d'organiser la rencontre entre acheteurs et vendeurs. Cette absence d'intermédiaire est rendue possible par l'exécution de contrats intelligents sur une blockchain publique.

Dans les marchés financiers traditionnels, les échanges sont généralement organisés autour de carnets d'ordres. Un carnet d'ordres recense les ordres d'achat et de vente déposés par les participants, puis les exécute selon des règles de priorité en prix et en temps. Un tel mécanisme est naturel dans un environnement centralisé, mais il est plus difficile à transposer directement sur une blockchain. En effet, maintenir un carnet d'ordres on-chain requiert de stocker un grand nombre d'ordres, de les mettre à jour fréquemment et d'exécuter une logique de correspondance potentiellement coûteuse. Ces contraintes sont particulièrement importantes sur Ethereum, où chaque opération de stockage ou de calcul entraîne des frais de transaction.

Les Automated Market Makers (AMM) apportent une réponse à cette difficulté. Plutôt que de faire correspondre directement des ordres d'achat et de vente, un AMM permet aux utilisateurs d'échanger contre des réserves de liquidité déposées dans un contrat intelligent. Ces réserves sont fournies par des agents appelés fournisseurs de liquidité. Le prix d'échange n'est pas déterminé par un carnet d'ordres, mais par une fonction mathématique reliant les quantités d'actifs disponibles dans le pool. Cette approche réduit fortement la complexité opérationnelle du mécanisme d'échange : l'état du marché est résumé par les réserves du pool, et l'exécution d'un échange revient à évaluer une fonction déterministe.

Les AMM les plus étudiés sont les *Constant Function Market Makers* (CFMM). Dans un CFMM, une transaction est acceptée lorsqu'elle préserve, à un ajustement de frais près, la valeur d'une fonction de trading appliquée aux réserves du pool. Le cas le plus célèbre est celui d'Uniswap V2, qui repose sur un invariant de produit constant. Ce modèle, malgré sa simplicité, permet de fournir de la liquidité de manière continue et entièrement automatisée.

1.1 DEX, limites des carnets d'ordres on-chain et rôle des AMM

Les DEX répondent à un problème central des marchés de crypto-actifs : permettre l'échange d'actifs sans dépendre d'une plateforme centralisée. Les plateformes centralisées peuvent offrir une forte liquidité et une faible latence, mais elles imposent aux utilisateurs de leur confier leurs fonds. Les DEX, au contraire, utilisent des contrats intelligents pour exécuter automatiquement les échanges et conserver les règles du marché de manière transparente.

Cependant, reproduire directement un carnet d'ordres sur une blockchain est coûteux. Un carnet d'ordres nécessite de stocker l'ensemble des ordres en attente, de les modifier lorsque les participants annulent ou actualisent leurs ordres, puis d'exécuter un algorithme de matching. Ces opérations sont coûteuses dans un environnement où chaque écriture en mémoire et chaque calcul sont facturés. C'est l'une des raisons pour lesquelles les AMM se sont imposés comme une architecture alternative pour les DEX.

Dans un AMM, les utilisateurs n'échangent pas directement entre eux. Ils échangent contre un pool de liquidité composé de plusieurs actifs. Le rôle du contrat intelligent est alors de déterminer, à partir des réserves disponibles, combien d'unités de l'actif sortant doivent être versées à l'utilisateur en échange de la quantité d'actif entrant. Cette règle est codée par une fonction de trading. Ainsi, le mécanisme d'échange est entièrement déterministe : à réserves données, un ordre donné produit une sortie donnée.

1.2 Le cas Uniswap V2 : invariant de produit constant

Le modèle le plus simple et le plus connu de CFMM est celui d'Uniswap V2. On considère un pool composé de deux actifs, notés α et β. Les réserves du pool sont notées $R_\alpha > 0$ et $R_\beta > 0$. Dans le modèle sans frais, le pool impose que le produit des réserves reste constant :

$$R_\alpha R_\beta = k.$$

Supposons qu'un utilisateur apporte une quantité $\Delta_\beta > 0$ de l'actif β afin de recevoir une quantité $\Lambda_\alpha > 0$ de l'actif α. Après l'échange, les réserves deviennent $R_\beta + \Delta_\beta$ et $R_\alpha - \Lambda_\alpha$. La condition de produit constant

s'écrit donc :

$$(R_\alpha - \Lambda_\alpha)(R_\beta + \Delta_\beta) = R_\alpha R_\beta.$$

Dans la pratique, Uniswap V2 applique des frais proportionnels sur l'actif entrant. Si l'on note $\gamma \in (0, 1]$ la fraction effectivement prise en compte dans l'invariant, la contrainte devient :

$$(R_\alpha - \Lambda_\alpha)(R_\beta + \gamma\Delta_\beta) = R_\alpha R_\beta.$$

En isolant Λ_α, on obtient la quantité reçue par l'utilisateur :

$$\Lambda_\alpha = \frac{R_\alpha \gamma \Delta_\beta}{R_\beta + \gamma\Delta_\beta}.$$

Cette formule met en évidence une propriété essentielle des AMM à produit constant : la sortie n'est pas linéaire en la taille de l'ordre. Plus la quantité échangée est grande relativement aux réserves du pool, plus l'impact de l'échange sur le prix est important.

1.3 Prix marginal, prix effectif et slippage

L'invariant de produit constant induit une distinction importante entre le prix marginal et le prix effectif d'un échange. Le prix marginal correspond au prix obtenu pour un échange infinitésimal. Dans le cas d'un pool à produit constant sans frais, le prix marginal de l'actif α exprimé en unités de β est donné par le rapport des réserves :

$$P^{\text{marg}}_{\alpha/\beta} = \frac{R_\beta}{R_\alpha}.$$

Ce prix marginal décrit la pente locale de la courbe d'échange. Il ne correspond cependant pas nécessairement au prix moyen effectivement payé lors d'un ordre de taille non négligeable. En effet, lorsqu'un utilisateur échange une quantité positive d'actif, les réserves du pool se modifient pendant l'exécution de l'ordre, ce qui déplace le prix le long de la courbe.

On appelle prix effectif le ratio entre la quantité reçue et la quantité fournie. Dans l'exemple précédent, où l'utilisateur fournit Δ_β et reçoit Λ_α, le prix effectif en unités de α reçues par unité de β fournie est :

$$P^{\text{eff}}(\Delta_\beta) = \frac{\Lambda_\alpha}{\Delta_\beta} = \frac{R_\alpha \gamma}{R_\beta + \gamma\Delta_\beta}.$$

Lorsque Δ_β augmente, ce prix effectif diminue. Ce phénomène est appelé *slippage*, ou glissement de prix. Il correspond à la perte relative liée au fait qu'un ordre de taille finie consomme une partie de la liquidité disponible. Le slippage est donc d'autant plus important que la taille de l'ordre est grande relativement aux réserves du pool.

Cette propriété explique pourquoi la liquidité joue un rôle central dans la qualité d'exécution. À prix marginal donné, un pool plus profond subira une variation de prix plus faible pour un même ordre. Inversement, un ordre important exécuté dans un pool peu liquide entraînera un slippage élevé.

1.4 Arbitrage local et efficacité des prix

Les prix internes d'un AMM ne sont pas fixés directement par un oracle externe. Ils résultent uniquement des réserves du pool. Il est donc possible que le prix marginal implicite d'un pool diverge temporairement du prix observé sur un marché de référence, par exemple une plateforme centralisée ou un autre DEX.

L'arbitrage joue alors un rôle essentiel. Si l'actif α est moins cher dans le pool que sur le marché de référence, un arbitrageur peut acheter α dans le pool puis le revendre sur le marché externe. Cette opération modifie les réserves du pool : la réserve de α diminue, celle de β augmente, et le prix marginal se rapproche du prix externe. Inversement, si α est trop cher dans le pool, l'arbitrageur réalise l'opération opposée. Par ce mécanisme, les arbitrageurs contribuent à aligner les prix des AMM sur les prix de marché.

La présence de frais empêche toutefois un alignement parfait. Dans un pool Uniswap avec un paramètre de frais γ, il existe une bande de non-arbitrage : tant que le prix interne reste suffisamment proche du prix externe, aucune transaction d'arbitrage n'est rentable après prise en compte des frais. Dans le cas d'un marché

de référence infiniment liquide, si l'on note m_p le prix de référence et m_u le prix implicite du pool sans frais, une condition de non-arbitrage prend la forme :

$$\gamma m_p \leq m_u \leq \gamma^{-1} m_p.$$

Cette inégalité illustre le rôle économique des frais : ils créent une zone dans laquelle les prix peuvent diverger sans donner naissance à une opportunité d'arbitrage exploitable. L'efficacité des prix dans un AMM doit donc être comprise relativement aux frictions de marché.

1.5 Du pool isolé au réseau de pools : motivation du routage optimal

L'analyse précédente concerne un pool isolé. Or, dans un écosystème DeFi réel, un utilisateur ne fait pas face à un seul pool, mais à un réseau de pools reliant de nombreux actifs. Par exemple, un échange d'ETH vers DAI peut être exécuté directement via un pool ETH/DAI, ou indirectement via une route ETH/USDC puis USDC/DAI. Il peut également être optimal de répartir l'ordre entre plusieurs routes afin de limiter le slippage sur chacune d'elles.

Cette observation conduit naturellement au problème du routage optimal. L'objectif n'est plus seulement de déterminer la sortie d'un swap dans un pool donné, mais de choisir un ensemble de transactions à travers plusieurs CFMM de manière à maximiser l'utilité finale du trader. Dans le cas le plus simple, cette utilité peut être la quantité finale d'un token cible reçue par l'utilisateur. Dans des formulations plus générales, elle peut représenter une préférence sur un panier final d'actifs.

Mathématiquement, un réseau de CFMM peut être représenté comme un graphe biparti composé d'un ensemble de pools et d'un ensemble de tokens. Un pool est relié aux tokens qu'il permet d'échanger. Chaque pool possède ses propres réserves, ses propres frais et sa propre fonction de trading. Une transaction globale est alors obtenue en agrégeant les flux locaux réalisés dans chaque pool.

Le problème central de ce mémoire est le suivant : comment formaliser et résoudre le problème de routage optimal dans un réseau de CFMM ? Nous verrons que, sous des hypothèses naturelles de concavité et de croissance des fonctions de trading, ce problème admet une relaxation convexe exacte. Cette propriété permet de résoudre efficacement le routage optimal et fournit également un cadre mathématique pour détecter l'existence d'opportunités d'arbitrage dans le réseau.

2 Modélisation Mathématiques d'un réseau de CFMM

Après avoir étudié le fonctionnement d'un pool isolé, nous passons maintenant à la modélisation d'un écosystème composé de plusieurs CFMM. Cette étape est nécessaire pour formuler rigoureusement le problème de routage optimal. En effet, dans un environnement DeFi réel, un utilisateur ne se limite pas à un seul pool de liquidité : il peut accéder simultanément à plusieurs pools, chacun reliant certains actifs et possédant ses propres réserves, frais et fonction de trading.

2.1 Réseau de m CFMM et n tokens

On considère un écosystème composé de m CFMM, indexés par

$$i = 1, \ldots, m,$$

et d'un ensemble global de n tokens, indexés par

$$j = 1, \ldots, n.$$

Chaque CFMM ne permet pas nécessairement d'échanger tous les tokens du réseau. On note donc n_i le nombre de tokens traités par le CFMM i, avec

$$2 \leq n_i \leq n.$$

Par exemple, un pool Uniswap V2 classique ne traite que deux tokens, tandis que certains protocoles plus généraux, comme Balancer, peuvent gérer des pools composés de plusieurs actifs.

Cette structure peut être représentée comme un graphe biparti. L'un des ensembles de sommets représente les CFMM, tandis que l'autre représente les tokens. Une arête relie un CFMM à un token si ce CFMM contient ce token dans ses réserves et permet donc de l'échanger. Cette représentation met en évidence le fait que l'écosystème ne doit pas être vu comme une collection indépendante de pools, mais comme un réseau de liquidité dans lequel les actifs sont reliés par plusieurs chemins possibles.

Par exemple, supposons que l'univers global des tokens soit

$$\{\text{WETH}, \text{USDC}, \text{DAI}, \text{WBTC}\}.$$

On peut considérer les quatre pools suivants :

$$\text{WETH/USDC}, \qquad \text{WETH/DAI}, \qquad \text{USDC/DAI}, \qquad \text{WETH/WBTC}.$$

Un utilisateur souhaitant échanger du WETH contre du DAI peut alors utiliser directement le pool WETH/DAI, mais il peut aussi emprunter une route indirecte passant par USDC :

$$\text{WETH} \longrightarrow \text{USDC} \longrightarrow \text{DAI}.$$

Il peut même être optimal de répartir l'ordre entre plusieurs routes afin de limiter l'impact de prix sur chaque pool. Cette observation est la motivation principale du routage optimal : il ne s'agit plus seulement de calculer la sortie d'un swap dans un pool donné, mais de déterminer comment utiliser au mieux l'ensemble du réseau de liquidité.

Dans la suite, nous cherchons donc à construire une notation permettant de décrire simultanément les transactions effectuées dans plusieurs CFMM. Pour cela, il faut distinguer deux niveaux d'indexation : l'indexation locale propre à chaque pool, et l'indexation globale commune à l'ensemble du réseau. Cette distinction sera formalisée à l'aide de matrices de projection dans la sous-section suivante.

Cette structure peut être visualisée sous la forme d'un graphe biparti reliant les tokens aux pools qui les contiennent, comme illustré par la figure 1.

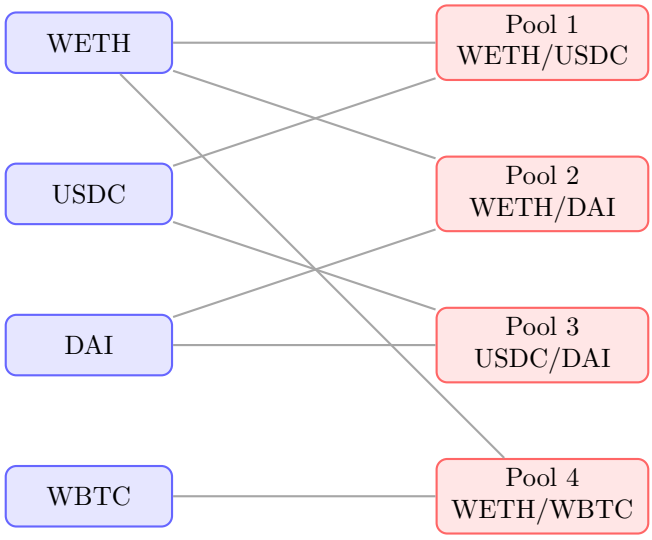

Figure 1 – Représentation bipartite d'un réseau de CFMM. Les sommets de gauche représentent les tokens, tandis que ceux de droite représentent les pools de liquidité. Une arête relie un token à un pool lorsque ce token appartient aux réserves du pool considéré.

Dans cet exemple, un échange de WETH vers DAI peut être exécuté directement via le pool WETH/DAI, ou indirectement en empruntant la route WETH $\to$ USDC $\to$ DAI. Cette multiplicité des chemins possibles justifie la nécessité d'un problème de routage optimal.

2.2 Indexation locale/globale et matrices A_i

Dans un réseau de CFMM, chaque pool possède sa propre indexation locale des tokens. En effet, le CFMM i ne traite qu'un sous-ensemble de n_i tokens parmi les n tokens disponibles dans l'écosystème global. Il est donc nécessaire de distinguer l'espace local du pool, de dimension n_i, et l'espace global du réseau, de dimension n.

L'espace global est commun à tous les CFMM. Il contient l'ensemble des tokens du réseau, indexés de 1 à n. Par exemple, dans l'univers

$$\{\mathrm{WETH}, \mathrm{USDC}, \mathrm{DAI}, \mathrm{WBTC}\},$$

on peut choisir l'indexation globale suivante :

$$\mathrm{WETH} \leftrightarrow 1, \qquad \mathrm{USDC} \leftrightarrow 2, \qquad \mathrm{DAI} \leftrightarrow 3, \qquad \mathrm{WBTC} \leftrightarrow 4.$$

Un vecteur global appartient alors à $\mathbb{R}^4$ et ses coordonnées correspondent toujours à ces quatre tokens dans cet ordre.

En revanche, chaque pool utilise seulement une partie de cet univers. Par exemple, le pool WETH/DAI ne contient que deux tokens. Son espace local est donc de dimension 2. Si l'on choisit localement l'ordre

$$(\mathrm{WETH}, \mathrm{DAI}),$$

alors un vecteur local du pool WETH/DAI appartient à $\mathbb{R}^2$. Pour pouvoir additionner les flux provenant de plusieurs pools, il faut donc convertir ces vecteurs locaux en vecteurs globaux.

Pour cela, on introduit, pour chaque CFMM i, une matrice

$$A_i \in \mathbb{R}^{n \times n_i}.$$

Cette matrice permet de projeter un vecteur exprimé dans les coordonnées locales du pool i vers les coordonnées globales du réseau. Plus précisément, si le k-ième token local du CFMM i correspond au j-ième token global, alors

$$(A_i)_{jk} = 1.$$

Dans tous les autres cas, on pose

$$(A_i)_{jk} = 0.$$

Ainsi, pour tout vecteur local $x_i \in \mathbb{R}^{n_i}$, le vecteur $A_i x_i \in \mathbb{R}^n$ est le même flux, mais exprimé dans l'espace global des tokens.

Reprenons l'exemple du pool WETH/DAI. Avec l'indexation globale

$$(\mathrm{WETH}, \mathrm{USDC}, \mathrm{DAI}, \mathrm{WBTC}),$$

et l'indexation locale du pool

$$(\mathrm{WETH}, \mathrm{DAI}),$$

la matrice associée au pool WETH/DAI est

$$A_{\mathrm{WETH/DAI}} = \begin{pmatrix} 1 & 0 \\ 0 & 0 \\ 0 & 1 \\ 0 & 0 \end{pmatrix}.$$

En effet, la première coordonnée locale correspond au WETH, qui est le premier token global, tandis que la deuxième coordonnée locale correspond au DAI, qui est le troisième token global.

Si l'on considère un vecteur local

$$x_i = \begin{pmatrix} x_{\mathrm{WETH}} \\ x_{\mathrm{DAI}} \end{pmatrix},$$

alors son expression globale est

$$A_{\mathrm{WETH/DAI}} x_i = \begin{pmatrix} x_{\mathrm{WETH}} \\ 0 \\ x_{\mathrm{DAI}} \\ 0 \end{pmatrix}.$$

Les coordonnées associées à USDC et WBTC sont nulles, car ces tokens ne sont pas présents dans ce pool.

Cette notation permet de comparer et d'agréger des transactions réalisées dans des pools différents. Elle sera essentielle pour définir le vecteur de transaction réseau, qui résume l'effet global de l'ensemble des échanges effectués par un trader à travers plusieurs CFMM.

2.3 Paniers fournis et reçus : Δ_i et Λ_i

Pour modéliser une transaction dans un CFMM, il ne suffit pas d'indiquer une paire d'actifs échangés. Dans le cadre général des CFMM, une transaction peut être décrite comme l'échange de deux paniers d'actifs : un panier fourni au pool et un panier reçu en retour.

Pour chaque CFMM i, on note

$$\Delta_i \in \mathbb{R}_+^{n_i}$$

le panier de tokens fourni au pool par le trader. La composante $(\Delta_i)_k$ représente la quantité du k-ième token local apportée au CFMM i.

De même, on note

$$\Lambda_i \in \mathbb{R}_+^{n_i}$$

le panier de tokens reçu par le trader en provenance du pool. La composante $(\Lambda_i)_k$ représente la quantité du k-ième token local retirée du CFMM i.

Ainsi, une transaction locale dans le CFMM i est représentée par le couple

$$(\Delta_i, \Lambda_i).$$

Les deux vecteurs sont supposés à composantes positives ou nulles, car ils décrivent des quantités physiques de tokens.

Dans le cas d'un pool Uniswap V2 composé de deux actifs, cette notation retrouve le cas usuel d'un swap simple. Par exemple, pour un pool WETH/DAI, si un utilisateur donne du WETH au pool afin de recevoir du DAI, on peut écrire localement

$$\Delta_i = \begin{pmatrix} \Delta_{\text{WETH}} \\ 0 \end{pmatrix}, \qquad \Lambda_i = \begin{pmatrix} 0 \\ \Lambda_{\text{DAI}} \end{pmatrix}.$$

Le trader fournit une quantité positive de WETH et reçoit une quantité positive de DAI. Les autres composantes sont nulles.

Cette notation est toutefois plus générale qu'un simple swap entre deux tokens. Elle permet aussi de représenter des échanges impliquant plusieurs actifs simultanément, ce qui est utile pour modéliser des CFMM multi-actifs. Par exemple, dans un pool contenant trois tokens, un trader pourrait fournir un panier composé de deux actifs et recevoir un troisième actif. La notation vectorielle permet donc de traiter de manière uniforme les pools à deux actifs et les pools multi-actifs.

Le gain net local du trader dans le CFMM i est alors donné par

$$\Lambda_i - \Delta_i.$$

Une composante positive signifie que le trader reçoit plus de ce token qu'il n'en fournit au pool. Une composante négative signifie au contraire qu'il fournit ce token au pool en quantité nette.

Cependant, ce vecteur net est encore exprimé dans l'indexation locale du CFMM i. Pour pouvoir agréger les effets de plusieurs transactions effectuées dans différents pools, on doit le convertir dans l'espace global du réseau à l'aide de la matrice A_i. Le vecteur

$$A_i(\Lambda_i - \Delta_i)$$

représente donc le gain net du trader dans le pool i, mais exprimé dans les coordonnées globales des n tokens.

Cette distinction entre flux locaux et flux globaux est fondamentale. Le trader peut par exemple recevoir de l'USDC dans un premier pool et fournir cet USDC dans un second pool. Localement, ces deux transactions apparaissent séparément ; globalement, elles peuvent se compenser. C'est précisément cette compensation des flux intermédiaires qui rend possible le routage multi-sauts.

Exemple. Supposons qu'un trader échange du WETH contre du DAI via la route indirecte

$$\text{WETH} \longrightarrow \text{USDC} \longrightarrow \text{DAI}.$$

Dans le premier pool, le trader fournit du WETH et reçoit de l'USDC. Dans le second pool, il fournit cet USDC et reçoit du DAI. L'USDC apparaît donc comme un actif intermédiaire : il est reçu localement dans un pool, puis fourni localement dans un autre. Dans le bilan global de la transaction, le trader ne cherche pas nécessairement à conserver de l'USDC ; seul le bilan net final sur l'ensemble des tokens est pertinent.

2.4 Fonction de trading φ_i et frais γ_i

Chaque CFMM est caractérisé par une fonction de trading, qui détermine les transactions acceptées par le pool. Pour le CFMM i, on note cette fonction

$$\varphi_i : \mathbb{R}_+^{n_i} \longrightarrow \mathbb{R}.$$

Elle est appliquée aux réserves du pool. On note

$$R_i \in \mathbb{R}_+^{n_i}$$

le vecteur des réserves courantes du CFMM i, exprimé dans son indexation locale.

Lorsqu'un trader propose une transaction locale (Δ_i, Λ_i), les réserves du pool sont modifiées. Le trader apporte le panier Δ_i au pool et reçoit le panier Λ_i en retour. En l'absence de frais, les réserves après transaction seraient donc

$$R_i + \Delta_i - \Lambda_i.$$

Cependant, les CFMM appliquent généralement des frais proportionnels sur les montants entrants. On introduit alors un paramètre

$$\gamma_i \in (0, 1],$$

qui représente la fraction du panier entrant effectivement prise en compte dans la fonction de trading. Par exemple, dans Uniswap V2, les frais sont de 0,3%, ce qui correspond à

$$\gamma_i = 0{,}997.$$

Les réserves pertinentes pour la contrainte de trading deviennent alors

$$R_i + \gamma_i \Delta_i - \Lambda_i.$$

La transaction est acceptée si elle ne diminue pas la valeur de la fonction de trading. On impose donc la contrainte

$$\varphi_i(R_i + \gamma_i \Delta_i - \Lambda_i) \geq \varphi_i(R_i).$$

Cette écriture signifie que, après prise en compte des frais et du panier retiré par le trader, le pool conserve au moins le même niveau d'invariant qu'avant la transaction.

Dans de nombreux AMM, la transaction est formulée comme une égalité :

$$\varphi_i(R_i + \gamma_i \Delta_i - \Lambda_i) = \varphi_i(R_i).$$

C'est par exemple le cas du modèle idéal d'Uniswap V2, où la fonction de trading est liée au produit des réserves. Toutefois, dans le cadre de l'optimisation convexe, il est plus naturel de travailler avec l'inégalité. Sous des hypothèses de croissance de la fonction de trading et de l'utilité du trader, cette relaxation sera saturée à l'optimum : le trader n'a pas intérêt à laisser de la valeur inutilisée dans le pool. Nous reviendrons sur ce point dans la section consacrée au problème de routage optimal.

Le modèle d'Uniswap V2 correspond au cas particulier d'un pool à deux actifs. Si le pool contient deux tokens, de réserves locales R_1 et R_2, la fonction de trading peut s'écrire, à une transformation monotone près,

$$\varphi(R_1, R_2) = \sqrt{R_1 R_2}.$$

Préserver cette fonction revient à préserver le produit des réserves :

$$R_1 R_2 = k.$$

Cette écriture sous forme de moyenne géométrique est particulièrement utile car elle met en évidence la concavité de la fonction de trading.

D'autres CFMM utilisent des fonctions de trading plus générales. Par exemple, un pool de type Balancer peut être modélisé par une moyenne géométrique pondérée :

$$\varphi(R) = \prod_{k=1}^{n_i} R_k^{w_k},$$

où les poids vérifient

$$w_k > 0, \qquad \sum_{k=1}^{n_i} w_k = 1.$$

Cette formulation généralise le produit constant à des pools contenant plusieurs actifs et des pondérations différentes.

Dans la suite, on supposera que les fonctions de trading φ_i sont croissantes et concaves. La croissance signifie qu'un pool préfère avoir davantage de réserves, toutes choses égales par ailleurs. La concavité traduit quant à elle la décroissance des rendements marginaux : plus un actif est retiré du pool relativement à ses réserves, plus il devient coûteux d'en retirer une unité supplémentaire. Ces deux hypothèses sont essentielles pour formuler le routage optimal comme un problème d'optimisation convexe.

2.5 Vecteur de transaction réseau Ψ

Les transactions décrites jusqu'ici sont locales : chaque couple (Δ_i, Λ_i) décrit uniquement ce qui se passe dans le CFMM i. Or, dans un problème de routage, le trader interagit simultanément avec plusieurs pools. Il faut donc agréger l'ensemble des flux locaux afin d'obtenir le bilan net global de la transaction.

Pour chaque CFMM i, le gain net local du trader est donné par

$$\Lambda_i - \Delta_i.$$

Ce vecteur appartient à $\mathbb{R}^{n_i}$, car il est exprimé dans les coordonnées locales du pool i. Pour l'exprimer dans l'espace global des n tokens, on utilise la matrice d'indexation A_i. Le gain net local, exprimé globalement, est donc

$$A_i(\Lambda_i - \Delta_i).$$

En sommant ces contributions sur l'ensemble des CFMM du réseau, on obtient le vecteur de transaction réseau :

$$\Psi = \sum_{i=1}^{m} A_i(\Lambda_i - \Delta_i).$$

Le vecteur $\Psi \in \mathbb{R}^n$ résume l'effet total de toutes les transactions réalisées par le trader à travers le réseau de CFMM. Sa j-ième composante représente la variation nette de la quantité du token global j dans le portefeuille du trader.

Ainsi :

$$\Psi_j > 0$$

signifie que le trader reçoit une quantité nette positive du token j à l'issue de l'ensemble des échanges. À l'inverse,

$$\Psi_j < 0$$

signifie que le trader fournit une quantité nette positive du token j au réseau. Enfin,

$$\Psi_j = 0$$

signifie que le token j est seulement utilisé comme actif intermédiaire, ou bien qu'il n'intervient pas dans la transaction.

Cette notation est particulièrement utile pour décrire les routes multi-sauts. Par exemple, considérons une transaction de WETH vers DAI passant par USDC :

$$\text{WETH} \longrightarrow \text{USDC} \longrightarrow \text{DAI}.$$

Dans le premier pool, le trader fournit du WETH et reçoit de l'USDC. Dans le second pool, il fournit de l'USDC et reçoit du DAI. Localement, l'USDC apparaît donc comme un token reçu dans un pool puis fourni dans un autre. Mais dans le bilan global, ces deux flux peuvent se compenser. Le vecteur Ψ permet précisément de ne retenir que le résultat net final : une quantité négative de WETH, une quantité positive de DAI, et éventuellement une composante nulle en USDC.

Par exemple, avec l'indexation globale

$$(\text{WETH}, \text{USDC}, \text{DAI}, \text{WBTC}),$$

une transaction idéale de WETH vers DAI pourrait avoir un vecteur réseau de la forme

$$\Psi = \begin{pmatrix} -\Delta_{\text{WETH}} \\ 0 \\ \Lambda_{\text{DAI}} \\ 0 \end{pmatrix}.$$

La composante WETH est négative, car le trader fournit du WETH au réseau. La composante DAI est positive, car il reçoit du DAI. La composante USDC est nulle, car l'USDC est seulement utilisé comme actif intermédiaire dans la route. La composante WBTC est également nulle, car ce token n'intervient pas dans cette transaction.

Le vecteur de transaction réseau permet donc de séparer deux niveaux d'analyse. Au niveau local, les variables (Δ_i, Λ_i) décrivent les échanges effectués dans chaque pool. Au niveau global, le vecteur Ψ décrit uniquement l'effet final sur le portefeuille du trader. C'est cette distinction qui rend possible la formulation du routage optimal comme un problème d'optimisation : les variables de décision sont les transactions locales, tandis que l'objectif du trader dépend du bilan global Ψ.

Le vecteur Ψ est l'objet central de la modélisation en réseau : il agrège les flux locaux de tous les CFMM et décrit la transaction nette finale du trader dans l'espace global des tokens.

2.6 Fonction d'utilité du trader

Le vecteur de transaction réseau Ψ décrit le bilan net final d'un ensemble de transactions à travers le réseau de CFMM. Cependant, il ne suffit pas à lui seul à déterminer si une transaction est souhaitable. Pour formuler un problème d'optimisation, il faut préciser les préférences du trader sur les bilans finaux possibles.

On introduit donc une fonction d'utilité

$$U : \mathbb{R}^n \longrightarrow \mathbb{R} \cup \{-\infty\},$$

qui associe à chaque vecteur de transaction réseau Ψ une valeur $U(\Psi)$. L'objectif du trader sera de choisir les transactions locales (Δ_i, Λ_i) de manière à maximiser cette utilité.

La fonction U permet de représenter différents types d'objectifs. Dans le cas le plus simple, un trader souhaite échanger une quantité donnée d'un token d'entrée contre la plus grande quantité possible d'un token de sortie. Par exemple, pour un échange de WETH vers DAI, on peut chercher à maximiser la composante DAI de Ψ, tout en imposant que la composante WETH corresponde à la quantité fournie au réseau. Si l'indexation globale est

$$(\text{WETH}, \text{USDC}, \text{DAI}, \text{WBTC}),$$

un objectif naturel consiste à maximiser

$$\Psi_{\text{DAI}},$$

sous la contrainte que

$$\Psi_{\text{WETH}} = -q,$$

où $q > 0$ désigne la quantité de WETH que le trader accepte de fournir.

Plus généralement, la fonction d'utilité peut représenter une préférence sur un panier final d'actifs. Par exemple, un trader peut attribuer une valeur externe à chaque token à l'aide d'un vecteur de prix

$$\pi \in \mathbb{R}^n_+,$$

et considérer une utilité linéaire

$$U(\Psi) = \pi^T \Psi.$$

Dans ce cas, le trader cherche à maximiser la valeur totale du bilan net de la transaction, exprimée dans le numéraire défini par π.

La fonction d'utilité peut également servir à encoder des contraintes. Par exemple, si le trader possède initialement un portefeuille

$$h \in \mathbb{R}^n_+,$$

il ne peut pas fournir plus de tokens qu'il n'en possède. Cette contrainte peut s'écrire

$$h + \Psi \geq 0.$$

On peut alors définir une fonction d'utilité qui vaut $-\infty$ lorsque cette contrainte n'est pas satisfaite. Ainsi, les transactions incompatibles avec les avoirs initiaux du trader sont automatiquement exclues du problème d'optimisation.

Dans le cadre du routage optimal, on suppose généralement que la fonction U est croissante et concave. La croissance signifie qu'un trader préfère recevoir davantage de tokens, toutes choses égales par ailleurs. La concavité traduit une forme d'aversion à la concentration ou, plus généralement, une décroissance de l'utilité marginale. Ces hypothèses sont également importantes d'un point de vue mathématique : elles permettent d'inscrire le problème de routage dans le cadre de l'optimisation convexe.

On dispose désormais de tous les objets nécessaires pour formuler le problème de routage optimal. Les variables de décision seront les paniers locaux (Δ_i, Λ_i) associés à chaque CFMM. Ces paniers devront respecter les contraintes de trading imposées par les fonctions φ_i. Le résultat global de ces transactions sera résumé par le vecteur Ψ, et l'objectif du trader sera de maximiser l'utilité $U(\Psi)$.

La section suivante consiste précisément à assembler ces éléments afin d'obtenir une formulation mathématique complète du problème de routage optimal.

3 Problème de routage optimal

La section précédente a introduit les objets nécessaires pour décrire un réseau de CFMM : les paniers locaux fournis et reçus dans chaque pool, les matrices d'indexation locale-globale, le vecteur de transaction réseau et la fonction d'utilité du trader. Nous pouvons maintenant assembler ces éléments afin de formuler le problème de routage optimal.

L'objectif est le suivant : étant donné un réseau de CFMM, des réserves courantes et une fonction d'utilité, déterminer les transactions locales à effectuer dans chaque pool afin de maximiser l'utilité finale du trader. Contrairement à une approche consistant à choisir simplement une route parmi plusieurs chemins possibles, la formulation en réseau autorise l'utilisation simultanée de plusieurs pools et la compensation des flux intermédiaires.

3.1 Formulation avec contraintes d'égalité

On considère un réseau composé de m CFMM et de n tokens. Pour chaque CFMM i, on note $R_i \in \mathbb{R}^{n_i}_+$ le vecteur des réserves courantes, φ_i sa fonction de trading, $\gamma_i \in (0, 1]$ son paramètre de frais, et $A_i \in \mathbb{R}^{n \times n_i}$ la matrice qui relie l'indexation locale du pool à l'indexation globale du réseau.

Les variables de décision sont les paniers locaux

$$\Delta_i \in \mathbb{R}^{n_i}_+, \qquad \Lambda_i \in \mathbb{R}^{n_i}_+, \qquad i = 1, \ldots, m,$$

ainsi que le vecteur de transaction réseau

$$\Psi \in \mathbb{R}^n.$$

Le panier Δ_i représente ce que le trader fournit au CFMM i, tandis que Λ_i représente ce qu'il reçoit de ce même CFMM.

Le vecteur global Ψ est obtenu en agrégeant les gains nets locaux :

$$\Psi = \sum_{i=1}^{m} A_i(\Lambda_i - \Delta_i).$$

Cette contrainte assure que Ψ correspond bien au bilan net de l'ensemble des transactions effectuées dans le réseau.

Chaque transaction locale doit également être valide pour le CFMM correspondant. Dans la formulation naturelle du problème, on impose donc que la fonction de trading soit exactement préservée :

$$\varphi_i(R_i + \gamma_i\Delta_i - \Lambda_i) = \varphi_i(R_i), \qquad i = 1, \ldots, m.$$

Cette contrainte signifie que, après prise en compte des frais et du panier retiré par le trader, le pool reste sur la même surface d'invariant.

Le problème de routage optimal peut alors s'écrire :

$$\begin{aligned}
\text{maximiser} \quad & U(\Psi) \\
\text{sous contraintes} \quad & \Psi = \sum_{i=1}^{m} A_i(\Lambda_i - \Delta_i), \\
& \varphi_i(R_i + \gamma_i\Delta_i - \Lambda_i) = \varphi_i(R_i), \qquad i = 1, \ldots, m, \\
& \Delta_i \geq 0, \quad \Lambda_i \geq 0, \qquad i = 1, \ldots, m.
\end{aligned}$$

Les données du problème sont les réserves R_i, les fonctions de trading φ_i, les frais γ_i, les matrices A_i et la fonction d'utilité U. Les variables sont les paniers Δ_i, Λ_i et le vecteur Ψ.

Cette formulation est très générale. Elle permet de décrire aussi bien un swap simple entre deux tokens qu'un routage multi-pools impliquant plusieurs actifs intermédiaires. Par exemple, dans un échange de WETH vers DAI, le trader peut fournir une quantité donnée de WETH, recevoir du DAI, et utiliser l'USDC comme actif intermédiaire sans que celui-ci apparaisse nécessairement dans le bilan final Ψ.

Cependant, le problème écrit sous cette forme n'est pas, en général, un problème d'optimisation convexe. La difficulté vient principalement des contraintes d'égalité imposant la préservation exacte des fonctions de trading. Lorsque φ_i est non affine, l'ensemble défini par

$$\varphi_i(R_i + \gamma_i\Delta_i - \Lambda_i) = \varphi_i(R_i)$$

n'est généralement pas convexe. Cette non-convexité rend la résolution directe du problème plus difficile.

L'idée centrale de la formulation moderne du routage optimal est alors de remplacer ces contraintes d'égalité par des contraintes d'inégalité. Cette relaxation transforme le problème en un problème convexe, tout en conservant la même valeur optimale sous des hypothèses naturelles. C'est l'objet de la sous-section suivante.

3.2 Relaxation convexe

La formulation précédente est naturelle, mais elle n'est généralement pas convexe à cause des contraintes d'égalité associées aux fonctions de trading. L'idée fondamentale consiste à remplacer ces égalités par des inégalités. Pour chaque CFMM i, on remplace donc

$$\varphi_i(R_i + \gamma_i\Delta_i - \Lambda_i) = \varphi_i(R_i)$$

par

$$\varphi_i(R_i + \gamma_i\Delta_i - \Lambda_i) \geq \varphi_i(R_i).$$

Cette nouvelle contrainte signifie que la transaction proposée ne doit pas diminuer la valeur de la fonction de trading du pool. Elle autorise donc, en apparence, des transactions qui laisseraient le pool avec une valeur d'invariant strictement supérieure à sa valeur initiale. On verra toutefois dans la sous-section suivante que cette relaxation est exacte sous des hypothèses naturelles : à l'optimum, les inégalités sont saturées.

Le problème relaxé s'écrit alors :

$$\begin{aligned}
\text{maximiser} \quad & U(\Psi) \\
\text{sous contraintes} \quad & \Psi = \sum_{i=1}^{m} A_i(\Lambda_i - \Delta_i), \\
& \varphi_i(R_i + \gamma_i \Delta_i - \Lambda_i) \geq \varphi_i(R_i), \qquad i = 1, \ldots, m, \\
& \Delta_i \geq 0, \quad \Lambda_i \geq 0, \qquad i = 1, \ldots, m.
\end{aligned}$$

Cette formulation possède une propriété essentielle : sous les hypothèses introduites précédemment, elle définit un problème d'optimisation convexe. En effet, on suppose que la fonction d'utilité U est concave et que les fonctions de trading φ_i sont concaves. Maximiser une fonction concave est compatible avec le cadre de l'optimisation convexe. Il reste donc à vérifier que les contraintes définissent un ensemble convexe.

La contrainte

$$\Psi = \sum_{i=1}^{m} A_i(\Lambda_i - \Delta_i)$$

est affine en les variables Ψ, Δ_i et Λ_i. Elle préserve donc la convexité.

Les contraintes de positivité

$$\Delta_i \geq 0, \qquad \Lambda_i \geq 0$$

sont également convexes, puisqu'elles définissent des orthants positifs.

Il reste à analyser la contrainte de trading relaxée :

$$\varphi_i(R_i + \gamma_i \Delta_i - \Lambda_i) \geq \varphi_i(R_i).$$

Le terme

$$R_i + \gamma_i \Delta_i - \Lambda_i$$

est affine en les variables Δ_i et Λ_i. Comme φ_i est concave, la fonction

$$(\Delta_i, \Lambda_i) \longmapsto \varphi_i(R_i + \gamma_i \Delta_i - \Lambda_i)$$

est concave. Or, pour une fonction concave f, l'ensemble de sur-niveau

$$\{x \mid f(x) \geq c\}$$

est convexe. Ainsi, la contrainte relaxée définit bien un ensemble convexe.

Le problème relaxé est donc un problème d'optimisation convexe : l'objectif est concave, et l'ensemble admissible est convexe. Cette propriété est déterminante, car elle permet d'utiliser des algorithmes standards d'optimisation convexe pour résoudre efficacement le routage optimal, même lorsque le réseau contient plusieurs pools et plusieurs tokens.

Cette relaxation transforme ainsi un problème a priori difficile, en raison des contraintes d'égalité non linéaires, en un problème numériquement tractable. La question restante est alors de savoir si cette transformation modifie réellement le problème économique initial. Autrement dit, les solutions du problème relaxé correspondent-elles encore à des transactions valides pour les CFMM ? La sous-section suivante montre que, sous l'hypothèse de croissance de l'utilité, la réponse est positive.

3.3 Exactitude de la relaxation

3.4 Exemple : liquidation d'un token vers un autre

3.5 Extension : coûts fixes et problème MICP

4 Arbitrage et équilibre du marché

4.1 Définition d'un arbitrage

4.2 Détection d'arbitrage par routage optimal

4.3 Conditions de non-arbitrage

4.4 Interprétation : prix globaux, prix locaux et bande de non-arbitrage

4.5 Limites économiques : frais, liquidité, gaz, exécution

5 Application empirique et backtesting sur données réelles

Bibliographie

[1] Auteur, *Titre*, revue ou éditeur, année.